The different routing methods in MoE's

Date: January 29, 2026

Andres Nowak [1]

1. Introduction

Why am I writing this? The reason I wrote this is that it seems we do not understand the balancing methods we use. The reason we want balancing is:

- First, we need to have computation shared uniformly across the expert parallelism group so as to use our resources correctly.
- Second, to use grouped GEMM effectively.
- And third, for performance, so as to help experts learn and not have some experts receive the majority of tokens from the beginning.

So, we always use this uniform balancing method, but uniform based on what exactly?

- The sequence?
- The global batch?
- The micro-batch?

Before, I assumed that batch-wise balancing was computed over the global batch, but it turns out that at least frameworks like Megatron-LM (Shoeybi et al., 2019) actually perform this at the micro-batch level by default. This means we are optimizing to distribute tokens evenly across experts within each individual micro-batch rather than across the full global batch.

Thinking about it now, this approach makes intuitive sense, since we only share tokens within an EP group and each rank in that EP group also functions as a separate DP rank. Optimizing per micro-batch ensures balanced distribution at the granularity where communication actually occurs. So, if we are running on just 4 GPUs with EP=4, we also effectively have DP=4. This means each rank in the EP group receives tokens from 4 different micro-batches in a single forward pass, yet the balancing loss constraint is still applied locally within each micro-batch.

But because we depend on the micro-batch, if one is doing micro-batch-based balancing, the question becomes what micro-batch size we should use. If one uses only a micro-batch of 1 or 2, there will not be any domain topic variance (or very little, even if we use intra-document attention masking) in the batch, and because of this we would not have the supposed *expert specialization*.

Also, what is the reason to use softmax over sigmoid in auxiliary-loss-free balancing? I understand we use sigmoid so as to give meaning to the bias and not have the experts fight for the router score. So we instead just use the sigmoid and then decide if we do a renormalization after the top-k.

2. DeepSeek V2

(DeepSeek-AI et al., 2024)

2.1. Routing method

$$
\begin{aligned}
h'_t &= u_t + \sum_{i=1}^{N_s} \mathrm{FFN}_i^{(s)}(u_t) + \sum_{i=1}^{N_r} g_{i,t} \mathrm{FFN}_i^{(r)}(u_t) \\
g_{i,t} &= \begin{cases} s_{i,t}, & s_{i,t} \in \mathrm{Topk}(\{s_{j,t} \mid 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise}, \end{cases} \\
s_{i,t} &= \mathrm{Softmax}_i(u_t^T e_i)
\end{aligned}
$$

This is the normal softmax routing method we know, and I think they do not do any renormalization after the top-k selection.

2.2. Balancing method

For the balancing method it was confusing to understand which one they use, as in the paper they say that they use expert balancing loss, but in Section 2.2.3 they then say T denotes the number of tokens in a sequence (not the total number of tokens). Also, based on the comparisons they do in (DeepSeek-AI et al., 2025) and its wording there, it seems DeepSeek-V2 and DeepSeek-MoE used sequence-wise balancing loss and not batch-wise auxiliary balancing loss.

So the sequence-wise balancing loss is defined as:

$$\begin{aligned}\mathcal{L}_{\text{aux_bal_seq}} &= \frac{1}{B} \cdot \frac{N_r}{k\,T_{\text{seq}}^2} \sum_{j=1}^{N_r \cdot B} (P_j\, f_j) \\ &= \frac{N_r}{k\,T_{\text{seq}}^2\,B} \sum_{j=1}^{N_r \cdot B} (P_j\, f_j)\,, \\ f_j &= \sum_{x \in T_{\text{seq}}} \text{routing_map}(x, j), \quad f_j \in \mathbb{N}_0^{N \cdot B} \\ P_j &= \sum_{x \in T_{\text{seq}}} \text{probs}(x, j), \quad P_j \in \mathbb{R}^{N \cdot B}\end{aligned}$$

where N_r are the routable experts and N_s are the shared ones and N is the total number of experts.

Besides the auxiliary balancing loss, they also use a device-balancing loss so as to be able to balance the work better between the GPUs in the Expert parallelism group.

3. DeepSeek-V3

(DeepSeek-AI et al., 2025)

3.1. Routing method

They use a sigmoid router instead of softmax for this one, because they say in (Wang et al., 2024) it gives better results and it makes the bias have meaning. With softmax you make the experts compete for the score, but with sigmoid each expert has its own score from 0 to 1 and we use the bias to balance which experts will be chosen in the top-k. The method is defined as:

$$\begin{aligned}h'_t &= u_t + \sum_{i=1}^{N_s} \text{FFN}_i^{(s)}(u_t) + \sum_{i=1}^{N_r} \lambda g_{i,t} \text{FFN}_i^{(r)}(u_t), \\ g_{i,t} &= \frac{g'_{i,t}}{\sum_{j=1}^{N_r} g'_{j,t}}, \\ g'_{i,t} &= \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j \mid 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise}, \end{cases} \\ s_{i,t} &= \text{Sigmoid}(u_t^T e_i),\end{aligned}$$

They do a renormalization after top-k to have the expert scores weighted by which one is more important to the token, but then these values are multiplied by a router scaling value that they do not mention in the paper. The router scale value they use is $\lambda = 2.5$.

3.2. Balancing method

They use auxiliary-loss-free with sequence-level auxiliary balancing loss.

- Auxiliary-loss-free, as it is implemented in libraries such as Megatron-LM, causes the same behavior as a global batch-wise auxiliary balancing loss. This is because you only need to do a DP reduction of the token counts per expert when doing the final global batch step update. So, the idea of the method is to add a bias to the scores that are given to each expert before top-k (and this bias will not be used in the weighted sum).

- Also they do a renormalization after the top-k, because the function they use is sigmoid and they need to convert the scores to a probability distribution to be able to use them in the auxiliary balancing loss.

$$\begin{aligned}L_{\text{aux_bal_seq}} &= \frac{1}{B} \sum_{i=1}^{N_r \cdot B} f_i P_i, \\ f_i &= \frac{N_r}{K_r T} \sum_{t=1}^{T} \mathbb{1}(s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j \mid 1 \leqslant j \leqslant N_r\}, K_r)), \\ s'_{i,t} &= \frac{s_{i,t}}{\sum_{j=1}^{N_r} s_{j,t}}, \\ P_i &= \frac{1}{T} \sum_{t=1}^{T} s'_{i,t}\end{aligned}$$

where $T = T_{\text{seq}}$.

And the auxiliary-free update formula that was used, from what I understand, is:

$$\begin{aligned}\bar{c}_i &= \frac{1}{N_r} \sum\nolimits_{i=1}^{N_r} c_i \\ e_i &= c_i - \bar{c}_i \\ \Delta b_i &= b_i + u \cdot \text{sign}(e_i) \\ b_i &= b_i + \Delta b_i\end{aligned}$$

where $b = [b_1, \ldots, b_{N_r}]$, $\overline{c_i}$ is the desired uniform number of tokens (what we want on average is uniform), and c_i is how many tokens were assigned to the expert. In the paper they mention this: "At the end of each step, we will decrease the bias term by γ if its corresponding expert is overloaded, and increase it by γ if its corresponding expert is underloaded, where γ is a hyper-parameter called bias update speed." And in (Wang et al., 2024) they say this was the best one.

In the paper they also compare against the batch-wise auxiliary balancing loss method:

$$\mathcal{L}_{\text{aux_bal_loss}} = \frac{E}{k\,(T_{\text{seq}} \cdot B)^2} \sum_{i=1}^{E} (P_i\, f_i)$$
$$f_i = \sum_{x \in (T_{\text{seq}} \cdot B)} \text{routing_map}(x, i), \quad f_i \in \mathbb{N}_0^E$$
$$P_i = \sum_{x \in (T_{\text{seq}} \cdot B)} \text{probs}(x, i), \quad P_i \in \mathbb{R}^E$$

They say that batch-wise auxiliary balancing loss gets the same performance or almost the same performance as the auxiliary-free method, but the difference is that in some batches or sequences there is an imbalance, or there can also be domain-shift imbalance at inference (likely because in a batch you have the data shuffled so you have many domains in a batch, but in inference you will likely only have one or maybe two domains).

They also show that auxiliary-free has more expert specialization, as they show that it has more hotspots in some datasets (like GitHub) for some experts (meaning some experts are used more than others), and this compared to the batch-wise auxiliary loss gets a uniform use distribution.

They also use node-limited routing. I understand it is so a device does not send its tokens to devices in very faraway nodes?

3.3. Questions

- What is the router scale value for? This is never mentioned in the DeepSeek-V3 paper or the auxiliary-loss-free paper.

- So basically the auxiliary-loss-free method can achieve almost perfect uniform distribution, but it can never converge to the desired value as mentioned in (Singh et al., 2026). This in the end causes a problem as we can get extreme imbalances inside a sequence, so it is necessary to use a sequence-wise balancing loss so as to enforce sequence-wise balancing so the model does not fail when we have only one batch or samples in a batch that have no topic variance.

4. Nvidia Nemotron 3

(NVIDIA et al., 2025)

For NVIDIA Nemotron 3 Nano they have 128 routable experts and 1 shared expert, with a Top-k $= 6$ and 52 layers.

4.1. Activation function in the MoE layer

They use ReLU^2 for the activation function in the experts.

4.2. Routing method

They use the same routing method as DeepSeek-V3 and Trinity with a router scale of $\lambda = 2.5$:

$$h'_t = u_t + \sum_{i=1}^{N_s} \text{FFN}_i^{(s)}(u_t) + \sum_{i=1}^{N_r} \lambda g_{i,t} \text{FFN}_i^{(r)}(u_t),$$
$$g_{i,t} = \frac{g'_{i,t}}{\sum_{j=1}^{N_r} g'_{j,t}},$$
$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} \mid 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise}, \end{cases}$$
$$s_{i,t} = \text{Sigmoid}(u_t^T e_i),$$

4.3. Balancing method

For the balancing in pre-training they use DeepSeek's aux-loss-free load balancing strategy (Wang et al., 2024; DeepSeek-AI, 2025b) with an update rate of 1×10^{-3} in conjunction with the standard load balancing loss. Here I do not know what they mean by the standard load balancing loss. In DeepSeek-V3 the balancing loss they use in conjunction is sequence-wise, but here they do not mention if it is sequence-wise or batch-wise.

In post-training they explicitly mention that they use sequence-wise balancing loss.

4.4. Questions

- Which auxiliary balancing method did they use in pre-training in conjunction with the aux-loss-free? I would think it is sequence-wise auxiliary balancing loss, because they just say they use a balancing loss, but it is not mentioned explicitly which one, as they do in the post-training part.

- Was the sequence-wise balancing loss used in post-training so as to not have to care about the batch size that is used? Because it seems the post-training of MoEs is very sensitive based on (Yang et al., 2025b).

5. Trinity

(Singh et al., 2026)

For the *mini* model they use 128 routable experts and 1 shared expert, with Top-$k = 8$. For the *nano* model the setup is the same: 128 routable experts and 1 shared expert, with Top-$k = 8$. For *Trinity-large* it has 400B parameters with 13B active, with 256 routable experts, 1 shared expert, Top-$k = 4$ (a very low one that causes a very high sparsity of $\approx 4\%$).

5.1. Routing method

For the routing method they use the same method as DeepSeek-V3, but the only difference is that they use a different scaling factor for the weighted sum compared to DeepSeek-V3, and it seems the value was maybe learned during training (like with a gain) as it is a very specific value.

For Trinity-nano (https://huggingface.co/arcee-ai/Trinity-Nano-Base/blob/main/config.json) they use $\lambda = 2.826$ as the routing scale value introduced in (DeepSeek-AI et al., 2025) (**well, in the code of the model, not in the paper at least**), and for Trinity-large (https://huggingface.co/arcee-ai/Trinity-Large-Base/blob/main/config.json) they use $\lambda = 2.448$.

$$
\begin{aligned}
h'_t &= u_t + \sum_{i=1}^{N_s} \mathrm{FFN}_i^{(s)}(u_t) + \sum_{i=1}^{N_r} \lambda g_{i,t} \mathrm{FFN}_i^{(r)}(u_t) \\
g_{i,t} &= \frac{g'_{i,t}}{\sum_{j=1}^{N_r} g'_{j,t}} \\
g'_{i,t} &= \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \mathrm{Topk}(\{s_{j,t} + b_j \mid 1 \leqslant j \leqslant N_r\}, K_r) \\ 0, & \text{otherwise,} \end{cases} \\
s_{i,t} &= \mathrm{Sigmoid}(u_t^T e_i),
\end{aligned}
$$

5.2. Balancing method

They use the auxiliary-loss-free method in conjunction with auxiliary balancing loss in sequence, as in (DeepSeek-AI et al., 2025), as we defined before in 3.2.

For Trinity Mini and Nano, the auxiliary-loss-free formula they use is:

$$
\begin{aligned}
\bar{c}_i &= \frac{1}{N_r} \sum\nolimits_{i=1}^{N_r} c_i \\
e_i &= c_i - \bar{c}_i \\
\Delta b_i &= b_i + u \cdot \mathrm{sign}(e_i) \\
b_i &= b_i + \Delta b_i \\
b_i &= b_i - \frac{1}{N_r} \sum_{j=1}^{N_r} b_j
\end{aligned}
$$

where $b = [b_1, \ldots, b_{N_r}]$, γ is the bias update rate, $\overline{c_i}$ is the desired uniform number of tokens (what we want on average is uniform), and c_i is how many tokens were assigned to the expert. From what I understand, the difference here compared to (DeepSeek-AI et al., 2025) is that they do mean centering for the bias update (or at least this is not mentioned in (DeepSeek-AI et al., 2025) and (Wang et al., 2024)).

For Trinity-large they decided to use another formula that they called **Soft-clamped Momentum Expert Bias Updates (SMEBU) load balancing**. The formula for this is:

$$
\begin{aligned}
v_i &= \frac{\bar{n} - n_i}{\bar{n}} \\
\tilde{v}_i &= \tanh(\kappa v_i) \\
\Delta b_i &= \lambda \tilde{v}_i \\
\Delta b_i &= \Delta b_i - \frac{1}{N_r} \sum_{j=1}^{N_r} \Delta b_j \\
m_i &= \beta m_i + (1 - \beta) \Delta b_i \\
b_i &= b_i + m_i
\end{aligned}
$$

So for this one, first instead of doing the $\pm\gamma$ bias update rate update, they first get normalized e_i, that is v_i (I think so as to have a more stable tanh computation), then they do $\tilde{v}_i = \tanh(\kappa v_i)$ where κ is a tunable scale (they say to control the saturation speed) and they use $\tanh$ to have a smooth approximation to $\pm$. Then they get $\Delta b_i = \lambda \tilde{v}_i$ with this new $\tilde{v}_i$, and they now call λ the load balance learning rate (but it can also still in a sense be called the bias update rate). Finally they mean-center Δb_i, update their momentum (or EMA) m_i, and use the momentum to update $b_i = b_i + m_i$.

They explain they did this change because they saw that with the normal auxiliary-loss-free method they had very unstable training (probably because the model is too sparse), and they note that this is because the auxiliary-loss-free training method can never converge to the perfect uniform distribution between experts, because the updates are just $\pm\lambda$. Quoting from them, they hypothesize that "this is a cause of instability for MoE training with the standard aux-loss-free objective. As the total number of experts increases, the per-layer norm of the bias step also increases. Viewing this as an optimization process, as this nears its local minima, the update step is biased towards oscillation, and can never fully settle."

6. OLMoE

For OLMoE they have 64 routable experts, Top-k $= 8$, and 16 layers.

6.1. Routing Method

Softmax routing without renormalization after the top-k:

$$o_t = \sum_{i=1}^{N} g_{i,t}\text{FFN}_i(u_t),$$
$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} \mid 1 \leqslant j \leqslant N\}, K), \\ 0, & \text{otherwise}, \end{cases}$$
$$s_{i,t} = \text{Softmax}_i(u_t^T e_i)$$

6.2. Balancing method

For the balancing method it seems they use batch-wise auxiliary balancing loss, but they do not mention if it is global batch-wise or micro-batch-wise. I would assume it is micro-batch-wise.

The mentioned formula in the paper is:

$$L_{LB} = N_E \cdot \sum_{i=1}^{N_E} f_i \cdot P_i$$

The code that they used seems to be this one, as referenced here: https://github.com/Muennighoff/megablocks/blob/4a25bc7b5665bcb9da93d72d5ad0c14d41e1a351/megablocks/layers/moe.py#L31

Looking at this, they have:

$$f_i = \frac{1}{KT} \sum_{x \in T} \text{routing_map}(x, i)$$
$$P_i = \frac{1}{T} \sum_{x \in T} \text{probs}(x, i)$$

where $T = \text{Batch_size} \cdot \text{Tokens}$, so it seems they use batch-wise sequence loss. And the reason the graph in the paper has as the minimum value something close to 0.1 instead of 1 is that it seems they visualize $w \cdot L_{LB}$ instead of L_{LB} (even though the aux loss coefficient w they use is 0.01 and not 0.1, but maybe we cannot see the value in the graph, so let us assume it is the correct formula).

P.S.: I asked *Muennighoff* (https://github.com/allenai/OLMoE/issues/45) (one of the authors of OLMoE (Muennighoff et al., 2025)) about the balancing loss that was used and what micro-batch size they used. For the balancing loss they say they use the one from Megablocks, and from what I could see the code is correct. For the micro-batch they said they use the biggest one they could fit, and from what I could see from their runs (https://wandb.ai/ai2-llm/olmoe/workspace?nw=nwuserakshitab) the biggest one was micro-batch of 64, so it seems this was the value they used.

6.2.1. Router Z-loss

They also use the router z-loss. The idea of this is that it penalizes large logits so as to have more stable training and better quality. I understand here logits means the values obtained from the linear projection of the router with the input tokens that come into the MoE layer.

$$L_{RZ}(x) = \frac{1}{B} \sum_{i=1}^{B} \left(\log \sum_{j=1}^{N_E} \exp(x_j^{(i)}) \right)^2$$

Right now, from what I understand, the router z-loss is more about penalizing the biggest logit. For example, if we have $\ln(e^{10} + e^5) = \ln(22026.47 + 148.41) = 10.0067$, then here what dominates is e^{10} as $\frac{22026.47}{22174.87} = 0.99$ is basically the majority of the mass, and the power we get is basically 10.

And at least in their experiments they showed that using z-loss had better performance in the loss and in some evals.

7. Qwen3-MoE

(Yang et al., 2025a)

For Qwen3-MoE 30B-A3B they use 128 routable experts, Top-k $= 8$, and 48 layers in total.

7.1. Balancing method

Qwen3-MoE is the first model that I have seen that uses a very different balancing method. Instead of doing sequence-wise or micro-batch-wise balancing loss, they decided to do the global batch-wise version (*this is the same as what aux-loss-free should be trying to achieve if your implementation does a DP reduction and accumulated sum over all micro-batches*). The problem with the global batch-wise auxiliary balancing loss is that you would need communication overhead to get the necessary values to do the loss computation, but the Qwen team designed a method that should add only $< 3\%$ latency overhead.

But first, let us again talk about micro-batch-wise load balancing. When training transformers we have multiple levels of parallelism (for now let us just work with data parallelism). In these parallelism strategies each GPU will only have a limited set of the data, and they say that existing framework implementations only compute the local f_i and P_i and then do an all-gather operation to average each contribution:

$$L_{\text{micro}}(\theta) = \frac{1}{N_p} \sum_{j=1}^{N_p} N_E \sum_{i=1}^{N_E} f_{i,j}(\theta) \cdot P_{i,j}(\theta)$$

where N_E is the number of experts and N_p is the number of GPUs in a parallelism group (again, let us just talk about data parallelism).

But the thing is, at least frameworks like Megatron-LM (Shoeybi et al., 2019) do not do this; they only do it for logging purposes. In the end what we care about is the gradient, and for the gradient an all-gather averaging operation for DP does happen (every framework from what I understand does this), so let us prove we get the same gradient:

Without the all-gather averaging operation in the backward, our balancing loss is only $L_j(\theta) = N_E \sum_{i=1}^{N_E} f_{i,j}(\theta) \cdot P_{i,j}(\theta)$, and the all-gather averaging of the gradients with respect to model parameter θ would be:

$$\frac{\partial L}{\partial \theta} = \frac{1}{N_P} \sum_{j=1}^{N_p} \frac{\partial L_j}{\partial \theta}$$

Because each GPU's local loss in the DP group contributes to the model parameter gradient. Equivalently, since $L_{\text{micro}}(\theta) = \frac{1}{N_p} \sum_{j=1}^{N_p} L_j(\theta)$, we have by linearity:

$$\frac{\partial L_{\text{micro}}}{\partial \theta} = \sum_{j=1}^{N_p} \frac{\partial L_{\text{micro}}}{\partial L_j} \cdot \frac{\partial L_j}{\partial \theta} = \sum_{j=1}^{N_p} \frac{1}{N_p} \cdot \frac{\partial L_j}{\partial \theta} = \frac{1}{N_p} \sum_{j=1}^{N_p} \frac{\partial L_j}{\partial \theta}$$

So both methods will give the same gradient of the loss. And if we also apply the DP gradient all-gather averaging to L_{micro} we would get:

$$\frac{1}{N_p} \sum_{k=1}^{N_p} \left(\frac{1}{N_p} \sum_{j=1}^{N_p} \frac{\partial L_j}{\partial \theta} \right)_k$$

And each GPU in the DP group will already have the same $\frac{1}{N_p} \sum_{j=1}^{N_p} \frac{\partial L_j}{\partial \theta}$ value, so in the end this would become:

$$\begin{aligned} &\frac{1}{N_p} \sum_{k=1}^{N_p} \left(\frac{1}{N_p} \sum_{j=1}^{N_p} \frac{\partial L_j}{\partial \theta} \right)_k \\ &= \frac{1}{N_p} N_p \left(\frac{1}{N_p} \sum_{j=1}^{N_p} \frac{\partial L_j}{\partial \theta} \right) \\ &= \frac{1}{N_p} \sum_{j=1}^{N_p} \frac{\partial L_j}{\partial \theta} \end{aligned}$$

P.S.: *For this part, I am not sure if I am correct, because if not then what Qwen did for their global batch-wise auxiliary balancing loss would be incorrect if we also applied the DP all-gather averaging (that I am pretty sure they do).*

7.1.1. Global batch-wise auxiliary balancing loss

So, if you know DP, you know that each GPU will only see a very small number of batches, and because of memory usage it is typical to have micro-batches of size ≤ 10 as the paper mentions (Lin et al., 2025). So, because of this, the experts will have to distribute uniformly among themselves a very limited set of domain tokens, and in the end probably many experts will see the same domain tokens. The theory of the Qwen team is that, based on their experiments and others like OLMoE (Muennighoff et al., 2025) and OpenMoE (Xue et al., 2024) (though they do not mention this one in their paper, it also demonstrates this), they see that what experts learn is just a simple token-ID-to-expert mapping.

Based on this, they decided to introduce a new global batch-wise auxiliary balancing loss:

$$L_{\text{aux_global}} = N_E \sum_{i=1}^{N_E} \bar{f}_i \cdot \bar{P}_i$$

where $\bar{f}_i$ and $\bar{P}_i$ would be equal to:

$$\begin{aligned} \bar{f}_i &= \frac{1}{K T_{\text{seq}} B N_p} \sum_{i=1}^{T_{\text{seq}} B N_p} \text{routing_map}(x, i) \\ &= \frac{1}{N_p} \sum_{j=1}^{N_p} \frac{1}{K T_{\text{seq}} B} \sum_{i=1}^{T_{\text{seq}} B} \text{routing_map}(x, i, j), \quad \bar{f}_i \in \mathbb{N}_0^E \\ \bar{P}_i &= \frac{1}{T_{\text{seq}} B N_p} \sum^{T_{\text{seq}} B N_p} \text{probs}(x, i) \\ &= \frac{1}{N_p} \sum_{j=1}^{N_p} \frac{1}{T_{\text{seq}} B} \sum_{i=1}^{T_{\text{seq}} B} \text{probs}(x, i, j), \quad \bar{P}_i \in \mathbb{R}^E \end{aligned}$$

where N_E is the number of experts, N_p is the number of parallelism groups, $\sum_{i=1}^{N_E} f_i = 1$, $\sum_{i=1}^{N_E} P_i = 1$, $B =$ micro_batch, and we say that $T = T_{\text{seq}} \cdot B$. From this, if we see it from the point of view of $\bar{f}_i$, then all j's from $f_{i,j}$ will multiply all of the j's in $P_{i,j}$. So, it is only possible to not have to do the all-gather averaging for only one of them (f_i or P_i), and in the paper (Lin et al., 2025) they decided to synchronize only f_i across all parallel groups to get $\bar{f}_i$. The final global batch-wise auxiliary balancing loss will be equal to the one we defined before:

$$
\begin{aligned}
L_{\text{aux_global}} &= N_E \sum_{i=1}^{N_E} \bar{f}_i \cdot \bar{P}_i \\
&= N_E \sum_{i=1}^{N_E} \bar{f}_i \cdot \left(\frac{1}{N_P} \sum_{j=1}^{N_P} P_i^j \right) \\
&= \frac{1}{N_P} \sum_{j=1}^{N_P} \left(N_E \sum_{i=1}^{N_E} \bar{f}_i \cdot P_i^j \right)
\end{aligned}
$$

and as we explained before in 7.1, we should be able to calculate the $\frac{1}{N_P}\sum_{j=1}^{N_P}$ from $L_{\text{aux_global}} = \frac{1}{N_P}\sum_{j=1}^{N_P}\left(N_E \sum_{i=1}^{N_E} \bar{f}_i \cdot P_i^j\right)$ with just the all-gather averaging from the gradients done in DP (if when talking about N_p we only talk about data parallelism).

They say that this method is good because "communicating only $f_i \in \mathbb{R}^{N_E}$ avoids the communication overhead of directly transmitting the token expert selection matrix of each parallel group and the expert selection scores (with a shape of tokens $\times$ experts)", because for f_i we only need to send a vector of size (experts).

7.1.2. Approximate the global batch-wise auxiliary balancing loss

Finally, because of gradient accumulation we in reality do not have the complete global batch from only N_p. So, for gradient accumulation they make an approximation using a buffer c_i for each expert to keep track of the new f_i for each expert.

7.2. Router method

The router method used is as follows:

$$
\begin{aligned}
o_t &= \sum_{i=1}^{N} g'_{i,t} \text{FFN}_i(u_t), \\
g'_{i,t} &= \frac{g_{i,t}}{\sum_j^E g_{j,t}} \\
g_{i,t} &= \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} \mid 1 \leqslant j \leqslant N\}, K), \\ 0, & \text{otherwise}, \end{cases} \\
s_{i,t} &= \text{Softmax}_i(u_t^T e_i)
\end{aligned}
$$

This is the same as OLMoE but we do a renormalization after the top-k, and this is equivalent to doing the top-k over the logits and doing softmax after the top-k:

$$
\begin{aligned}
o_t &= \sum_{i=1}^{N} g'_{i,t} \text{FFN}_i(u_t), \\
g'_{i,t} &= \frac{g_{i,t}}{\sum_j^E g_{j,t}} \\
g_{i,t} &= \begin{cases} l_{i,t}, & l_{i,t} \in \text{Topk}(\{l_{j,t} \mid 1 \leqslant j \leqslant N\}, K), \\ 0, & \text{otherwise}, \end{cases} \\
l_{i,t} &= u_t^T e_i
\end{aligned}
$$

This is equivalent because:

$$
\begin{aligned}
p_i^{(B)} &= \frac{\frac{\exp(z_i)}{\sum_{l=1}^n \exp(z_l)}}{\sum_{j \in S} \frac{\exp(z_j)}{\sum_{l=1}^n \exp(z_l)}} \\
&= \frac{\exp(z_i)}{\sum_{l=1}^n \exp(z_l)} \div \frac{\sum_{j \in S} \exp(z_j)}{\sum_{l=1}^n \exp(z_l)} \\
&= \frac{\exp(z_i) \cdot \sum_{l=1}^n \exp(z_l)}{\sum_{l=1}^n \exp(z_l) \cdot \sum_{j \in S} \exp(z_j)} \\
&= \frac{\exp(z_i)}{\sum_{j \in S} \exp(z_j)} \\
&= p_i^{(A)}
\end{aligned}
$$

where $p_i^{(A)}$ is the probability if we did softmax after the top-k instead, and S means being selected in the top-k.

Also, in Qwen3-MoE they do not use shared experts, but in Qwen3-next they decided to use them and they still use the renormalization after the top-k selection.

8. Opinions

- What I want to understand from this method is whether in reality it is better to do the global batch-wise auxiliary balancing loss instead of only the micro-batch-wise auxiliary balancing loss. In their paper, they show two things:
  - First they show an advantage of $\approx 2\%$ across Hellaswag and MMLU when comparing L_{micro} and L_{global} using a micro-batch size of 2 and 4. But this is such a small micro-batch size; at least it would be interesting to see a comparison with one of 16 and also see if just doing different seeds would have given the same performance difference. But they do at least show that doing a shuffle from all the micro-batches (to do an approximation of the global batch) gives approximately the same performance as their L_{global}.
  - They also show a perplexity graph reduction going from micro-batch size of 2 to 512, training on

400B tokens for an MoE 3.4B-A0.6B. The difference between the two was 0.185 PPL, but I would have also liked to see downstream task difference in precision. Also in this experiment they show that going from Top-k $= 4$ to Top-k $= 6$ was the same as going from micro-batch size of 2 to 8 for L_{micro}.

- Finally they also show an expert-to-token ratio usage comparing L_{micro} and L_{global}, and they show that in different domain datasets some experts are used more than others when using L_{global}, while for L_{micro} there is basically no expert that is used more than others. And I think the expert-to-token ratio usage is defined here as how many times E_i is selected for token t divided by how many times token t appears across the whole dataset.

9. Pangu Pro MoE

This model is one that had a lot of controversy, as this model was said to have been an upcycled model from Qwen, but even if we cannot confirm how good the model was in reality, the grouped MoE idea is still interesting to talk about at least.

9.1. Router method

Here let us first explain the routing method. The method of routing they have is, first we have:

$$S = \text{Softmax}\left(W^\top h\right)$$

Here S is a vector of length N (meaning the number of experts), where S_i is the initial score for expert E_i, W is the router weight matrix $\mathbb{R}^{d\times n}$, and h is the input token.

So here in Pangu Pro MoE, the idea is that we will divide this vector among a group of size M (here the idea is about the expert parallelism). So we will have S_j contain the scores of $N_g = N/M$ experts belonging to group j (the idea is to have the same N_g in all groups). Now based on this we do the top-k$'$ selection among each group, so N_g instead of across the N experts. So here we have a total K but in each group we do a selection of size K' and the gate scores will be equal to:

$$G'(h) = \text{concat}(\text{Topk}(S_1, K'), \text{Topk}(S_2, K'), \ldots, \text{Topk}(S_M, K'))$$

These are the scores that will be used in the weighted sum:

$$y_{\text{MoGE}} = \sum_{i=1}^{N} G'(h)_i \cdot E_i(h)$$

9.2. Balancing method

As you can see, MoGE (that is what they called their method) solves the problem of inter-group load balance (i.e., the expert parallelism across devices), but we still need to ensure that the routing mechanism distributes the workload reasonably among experts. For one, we still have the grouped GEMM problem (as is also mentioned in SonicMoE), but we also need to be able to let all experts learn something and not have some overused experts among each group. So for this we have:

$$\ell_{\text{aux}} = \sum_{i=1}^{N} f_i p_i$$

where:

$$f_i = \frac{N}{KT}\sum_{t\in T}\mathbb{I}\{\text{Token } t \text{ selects Expert } i\}, \quad p_i = \frac{1}{T}\sum_{t\in T} S_{i,t}$$

where $T = \text{Batch-Size} \cdot T_{\text{seq}}$.

Now, one would think here that f_i is wrong because f_i is decided among N_g and K', not N and K, but we have this: first we know that $N = M \cdot N_g$ and $K = M \cdot K'$, so we have:

$$\frac{N}{KT} = \frac{MN_g}{MK'T} = \frac{N_g}{K'T}$$

So in the end we have the correct thing, and if we remove N from f_i and put it in the auxiliary balancing loss directly instead, we have that $\sum_{i=1}^{E} f_i = 1$, and because of this it is still possible for p_i to approximate what f_i will have and be able to get to the minimum loss.

9.3. Opinions on this method

- One thing you might think when you see this is: are we not constrained on the expert-parallelism side at inference time? The number of groups in MoGE is basically about getting a perfect division of experts across devices, so the work is always split evenly, no matter what.
  - But what if we want to go beyond the expert parallelism used in training at inference time? I think one can still support a larger max EP. For example, if we train with $EP = 4$ but want to support $EP = 8$, we can do it by dividing each group on each device by 2 at training time. Then, each device in the EP group at training time would have 2 groups and the MoGE method is still maintained by doing this.
- The second thing is that I do not know if doing auxiliary balancing loss over all N experts makes sense. Because in this method we already solved part of the balancing problem for performance by forcing all tokens to go to each device in the EP group. So now it is about balancing the tokens inside the EP group for the grouped GEMM computation and being able

to have all experts in the group learn something. So I feel here using auxiliary-loss-free training could be better, as we would not affect directly the router scores for each expert and we let them learn what is necessary for the weighted sum of the experts (as the probability distribution is already made over all the experts of the model).

10. LongCat-flash

(Team et al., 2025)

In September 2025, the LongCat team implemented a model using zero-computation experts alongside FFN experts.

10.1. Routing method

LongCat-flash uses a standard top-k routing mechanism where tokens are routed to either FFN experts or zero-computation experts. However, unlike typical zero-computation experts that output zero, their zero experts implement the identity function:

$$\text{expert_output} = \begin{cases} \text{FFN}(x_i), & \text{if expert} \in \mathcal{S}_{\text{FFN}}, \\ x_i, & \text{if expert} \in \mathcal{S}_{\text{zero}}. \end{cases}$$

The idea for using identity rather than zero $f : x \to 0$ is likely to provide gradients for the router to learn something about them, because they do not use the standard auxiliary balancing loss (relying instead on device-level balancing loss and the bias updates). However, we think this creates redundancy with the residual connection, as the identity of the token will already be added via the residual. Consequently, the token representation receives the identity contribution multiple times, once from the residual connection and in a weighted manner from each of the zero-computation experts in the weighted sum of a token.

Additionally, they use a lot of zero-computation experts (even though all of them are the same thing). Specifically, they use 512 FFN experts and 256 zero-computation experts, despite only selecting $K = 12$ experts per token. We think this was done because it helped balance the distribution among experts and made it easier for the router to converge to the correct balance distribution.

10.2. Balancing method

They employ the auxiliary-loss-free bias method from (Wang et al., 2024) (without sequence-wise balancing loss, unlike in (DeepSeek-AI et al., 2025), but they still use a device-level balancing loss like in (DeepSeek-AI et al., 2025)), with a modified update rule that accounts for the expected number of FFN experts:

$$\Delta b_i = \begin{cases} \mu \left(\frac{K_e}{K} \cdot \frac{1}{N} - \frac{T_i}{K\, T_{\text{all}}} \right), & \text{if } 1 \le i \le N, \\ 0, & \text{if } N < i \le N + Z. \end{cases}$$

where μ is the bias adaptation rate, T_{all} is the total number of tokens in the global batch, K_e is the expected number of FFN experts actually used (which is lower than K), N is the number of FFN experts, and Z is the number of zero-computation experts.

While standard uniform balancing targets $\frac{1}{N}$ for each expert, here the target is adjusted for the expected FFN utilization:

$$\frac{K_e}{NK}$$

instead of $\frac{1}{N}$. The bias is updated as $b_i = b_i + \Delta b_i$, with no update for zero experts ($\Delta b_i = 0$ for $i > N$) since they serve as a default destination. This bias is added to the router scores for the top-k selection:

$$s_i = x \cdot W_{g,i} + b_i$$

For their configuration, they use $K_e = 8$, $K = 12$, and **they use a PID controller for the bias update** rather than a simple sign-based bias update rule like in (DeepSeek-AI et al., 2025).

11. Extra

- Megatron-LM by default for aux_balancing loss (not sequence or global) uses the micro-batch size (not global one), but this makes sense in a way, as expert parallelism is per DP so in a sense there is no point in balancing over the DP level as the speed and imbalance is only affected at the EP level. But because of this one has to be careful what micro-batch size they use, as having a micro-batch size of 1 means that probably we will not have any domain variety in a batch, so the experts here will not learn any specialization.
  - But now seeing the Qwen3-MoE method, maybe it can make sense to do the global auxiliary balancing loss.
- Also Megatron does do a reduction over DP but only for logging purposes (and it does it at the end of the model before doing the logging of that forward pass).
- It should be possible to increase the batch size used in aux_loss with EP, because we already do a connection between ranks in the EP group, so it should be possible to get information from each one if we want to make the aux_loss uniform across the EP group instead of a single DP micro-batch.

- The implementation for aux-loss-free in Megatron-LM is a global batch optimization operation for the balancing, as it does a running sum and a reduction in the DP world to get the expert token count (so how many times an expert was chosen).

12. Revision

- Before I had that aux-loss-free was a micro-batch-wise optimization, but I was wrong. At least the implementation in Megatron-LM does it so it is a global batch-wise, and it makes sense that this is possible because you only need one DP communication and it is a very cheap one because the size of the tensor is just the number of experts.
- Used Kimi 2.6 to make corrections on types and improve clarity.

References

Arcee AI. The trinity manifesto, 2026. URL https://www.arcee.ai/blog/the-trinity-manifesto. Blog post; accessed 2026-01-27.

DeepSeek-AI, Liu, A., Feng, B., Wang, B., Wang, B., Liu, B., Zhao, C., Dengr, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Yang, H., Zhang, H., Ding, H., Xin, H., Gao, H., Li, H., Qu, H., Cai, J. L., Liang, J., Guo, J., Ni, J., Li, J., Chen, J., Yuan, J., Qiu, J., Song, J., Dong, K., Gao, K., Guan, K., Wang, L., Zhang, L., Xu, L., Xia, L., Zhao, L., Zhang, L., Li, M., Wang, M., Zhang, M., Zhang, M., Tang, M., Li, M., Tian, N., Huang, P., Wang, P., Zhang, P., Zhu, Q., Chen, Q., Du, Q., Chen, R. J., Jin, R. L., Ge, R., Pan, R., Xu, R., Chen, R., Li, S. S., Lu, S., Zhou, S., Chen, S., Wu, S., Ye, S., Ma, S., Wang, S., Zhou, S., Yu, S., Zhou, S., Zheng, S., Wang, T., Pei, T., Yuan, T., Sun, T., Xiao, W. L., Zeng, W., An, W., Liu, W., Liang, W., Gao, W., Zhang, W., Li, X. Q., Jin, X., Wang, X., Bi, X., Liu, X., Wang, X., Shen, X., Chen, X., Chen, X., Nie, X., Sun, X., Wang, X., Liu, X., Xie, X., Yu, X., Song, X., Zhou, X., Yang, X., Lu, X., Su, X., Wu, Y., Li, Y. K., Wei, Y. X., Zhu, Y. X., Xu, Y., Huang, Y., Li, Y., Zhao, Y., Sun, Y., Li, Y., Wang, Y., Zheng, Y., Zhang, Y., Xiong, Y., Zhao, Y., He, Y., Tang, Y., Piao, Y., Dong, Y., Tan, Y., Liu, Y., Wang, Y., Guo, Y., Zhu, Y., Wang, Y., Zou, Y., Zha, Y., Ma, Y., Yan, Y., You, Y., Liu, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Huang, Z., Zhang, Z., Xie, Z., Hao, Z., Shao, Z., Wen, Z., Xu, Z., Zhang, Z., Li, Z., Wang, Z., Gu, Z., Li, Z., and Xie, Z. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024. URL https://arxiv.org/abs/2405.04434.

DeepSeek-AI, Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Bao, H., Xu, H., Wang, H., Zhang, H., Ding, H., Xin, H., Gao, H., Li, H., Qu, H., Cai, J. L., Liang, J., Guo, J., Ni, J., Li, J., Wang, J., Chen, J., Chen, J., Yuan, J., Qiu, J., Li, J., Song, J., Dong, K., Hu, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Xu, L., Xia, L., Zhao, L., Wang, L., Zhang, L., Li, M., Wang, M., Zhang, M., Zhang, M., Tang, M., Li, M., Tian, N., Huang, P., Wang, P., Zhang, P., Wang, Q., Zhu, Q., Chen, Q., Du, Q., Chen, R. J., Jin, R. L., Ge, R., Zhang, R., Pan, R., Wang, R., Xu, R., Zhang, R., Chen, R., Li, S. S., Lu, S., Zhou, S., Chen, S., Wu, S., Ye, S., Ye, S., Ma, S., Wang, S., Zhou, S., Yu, S., Zhou, S., Pan, S., Wang, T., Yun, T., Pei, T., Sun, T., Xiao, W. L., Zeng, W., Zhao, W., An, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Li, X. Q., Jin, X., Wang, X., Bi, X., Liu, X., Wang, X., Shen, X., Chen, X., Zhang, X., Chen, X., Nie, X., Sun, X., Wang, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yu, X., Song, X., Shan, X., Zhou, X., Yang, X., Li, X., Su, X., Lin, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhu, Y. X., Zhang, Y., Xu, Y., Xu, Y., Huang, Y., Li, Y., Zhao, Y., Sun, Y., Li, Y., Wang, Y., Yu, Y., Zheng, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Tang, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Wu, Y., Ou, Y., Zhu, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Zha, Y., Xiong, Y., Ma, Y., Yan, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Wu, Z. F., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Huang, Z., Zhang, Z., Xie, Z., Zhang, Z., Hao, Z., Gou, Z., Ma, Z., Yan, Z., Shao, Z., Xu, Z., Wu, Z., Zhang, Z., Li, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Gao, Z., and Pan, Z. Deepseek-v3 technical report, 2025. URL https://arxiv.org/abs/2412.19437.

Guo, W., Mishra, M., Cheng, X., Stoica, I., and Dao, T. Sonicmoe: Accelerating moe with io and tile-aware optimizations, 2025. URL https://arxiv.org/abs/2512.14080.

Lin, J., Zhou, J., Titov, I., Qiu, Z., Liu, D., Men, R., Wang, Z., Zheng, B., Wen, K., and Huang, Z. Demons in the detail: On implementing load balancing loss for training specialized mixture-of-expert models, 2025. URL https://arxiv.org/abs/2501.11873.

Muennighoff, N., Soldaini, L., Groeneveld, D., Lo, K., Morrison, J., Min, S., Shi, W., Walsh, P., Tafjord, O., Lambert, N., Gu, Y., Arora, S., Bhagia, A., Schwenk, D., Wadden, D., Wettig, A., Hui, B., Dettmers, T., Kiela, D., Farhadi, A., Smith, N. A., Koh, P. W., Singh, A., and Hajishirzi, H. Olmoe: Open mixture-of-experts language models, 2025. URL https://arxiv.org/abs/2409.02060.

NVIDIA, :, Blakeman, A., Grattafiori, A., Basant, A., Gupta,

A., Khattar, A., Renduchintala, A., Vavre, A., Shukla, A., Bercovich, A., Ficek, A., Shaposhnikov, A., Kondratenko, A., Bukharin, A., Milesi, A., Taghibakhshi, A., Liu, A., Barton, A., Mahabaleshwarkar, A. S., Klein, A., Zuker, A., Geifman, A., Shen, A., Bhiwandiwalla, A., Tao, A., Guan, A., Mandarwal, A., Mehta, A., Aithal, A., Poojary, A., Ahamed, A., Thekkumpate, A. K., Dattagupta, A., Zhu, B., Sadeghi, B., Simkin, B., Lanir, B., Schifferer, B., Nushi, B., Kartal, B., Rouhani, B. D., Ginsburg, B., Norick, B., Soubasis, B., Kisacanin, B., Yu, B., Catanzaro, B., del Mundo, C., Hwang, C., Wang, C., Hsieh, C.-P., Zhang, C., Yu, C., Mungekar, C., Patel, C., Alexiuk, C., Parisien, C., Neale, C., Mosk-Aoyama, D., Su, D., Corneil, D., Afrimi, D., Rohrer, D., Serebrenik, D., Gitman, D., Levy, D., Stosic, D., Mosallanezhad, D., Narayanan, D., Nathawani, D., Rekesh, D., Yared, D., Kakwani, D., Ahn, D., Riach, D., Stosic, D., Minasyan, E., Lin, E., Long, E., Long, E. P., Lantz, E., Evans, E., Ning, E., Chung, E., Harper, E., Tramel, E., Galinkin, E., Pounds, E., Briones, E., Bakhturina, E., Ladhak, F., Wang, F., Jia, F., Soares, F., Chen, F., Galko, F., Siino, F., Agam, G. H., Ajjanagadde, G., Bhatt, G., Prasad, G., Armstrong, G., Shen, G., Batmaz, G., Nalbandyan, G., Qian, H., Sharma, H., Ross, H., Ngo, H., Sahota, H., Wang, H., Soni, H., Upadhyay, H., Mao, H., Nguyen, H. C., Nguyen, H. Q., Cunningham, I., Shahaf, I., Gitman, I., Loshchilov, I., Moshkov, I., Putterman, I., Kautz, J., Scowcroft, J. P., Casper, J., Mitra, J., Glick, J., Chen, J., Oliver, J., Zhang, J., Zeng, J., Lou, J., Zhang, J., Huang, J., Conway, J., Guman, J., Kamalu, J., Greco, J., Cohen, J., Jennings, J., Daw, J., Vialard, J. V., Yi, J., Parmar, J., Xu, K., Zhu, K., Briski, K., Cheung, K., Luna, K., Santhanam, K., Shih, K., Kong, K., Bhardwaj, K., Puvvada, K. C., Pawelec, K., Anik, K., McAfee, L., Sleiman, L., Derczynski, L., Ding, L., Liebenwein, L., Vega, L., Grover, M., Segbroeck, M. V., de Melo, M. R., Sreedhar, M. N., Kilaru, M., Ashkenazi, M., Romeijn, M., Cai, M., Kliegl, M., Moosaei, M., Novikov, M., Samadi, M., Corpuz, M., Wang, M., Price, M., Boone, M., Evans, M., Martinez, M., Chrzanowski, M., Shoeybi, M., Patwary, M., Mulepati, N., Hereth, N., Assaf, N., Habibi, N., Zmora, N., Haber, N., Sessions, N., Bhatia, N., Jukar, N., Pope, N., Ludwig, N., Tajbakhsh, N., Juluru, N., Hrinchuk, O., Kuchaiev, O., Delalleau, O., Olabiyi, O., Argov, O. U., Xie, O., Chadha, P., Shamis, P., Molchanov, P., Morkisz, P., Dykas, P., Jin, P., Xu, P., Januszewski, P., Thombre, P. P., Varshney, P., Gundecha, P., Miao, Q., Mahabadi, R. K., El-Yaniv, R., Zilberstein, R., Shafipour, R., Harang, R., Izzo, R., Shahbazyan, R., Garg, R., Borkar, R., Gala, R., Islam, R., Waleffe, R., Watve, R., Koren, R., Zhang, R., Hewett, R. J., Prenger, R., Timbrook, R., Mahdavi, S., Modi, S., Kriman, S., Kariyappa, S., Satheesh, S., Kaji, S., Pasumarthi, S., Narentharen, S., Narenthiran, S., Bak, S., Kashirsky, S., Poulos, S., Mor, S., Ramasamy, S., Acharya, S., Ghosh, S., Sreenivas, S. T., Thomas, S., Fan, S., Gopal, S., Prabhumoye, S., Pachori, S., Toshniwal, S., Ding, S., Singh, S., Sun, S., Ithape, S., Majumdar, S., Singhal, S., Alborghetti, S., Ge, S., Devare, S. D., Barua, S. K., Panguluri, S., Gupta, S., Priyadarshi, S., Akter, S. N., Bui, T., Ene, T.-D., Kong, T., Do, T., Blankevoort, T., Balough, T., Asida, T., Natan, T. B., Konuk, T., Vashishth, T., Karpas, U., De, U., Noorozi, V., Noroozi, V., Srinivasan, V., Elango, V., Korthikanti, V., Kurin, V., Lavrukhin, V., Jiang, W., Ahmad, W. U., Du, W., Ping, W., Zhou, W., Jennings, W., Zhang, W., Prazuch, W., Ren, X., Karnati, Y., Choi, Y., Meyer, Y., Wu, Y.-F., Zhang, Y., Lin, Y., Geifman, Y., Fu, Y., Subara, Y., Suhara, Y., Gao, Y., Moshe, Z., Dong, Z., Liu, Z., Chen, Z., and Yan, Z. Nemotron 3 nano: Open, efficient mixture-of-experts hybrid mamba-transformer model for agentic reasoning, 2025. URL https://arxiv.org/abs/2512.20848.

Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

Singh, V., Krauss, L., Jaghouar, S., Sirovatka, M., Goddard, C., Obeid, F., Ong, J. M., Straube, J., Fern, Harley, A., Stewart, C., Kealty, C., Panahi, M., Kirsten, S., Deshpande, A., Vij, A., Bresnu, A., Veldurthi, P., Ravishankar, R., Bishnoi, H., McQuade, M., Hagemann, J., and Atkins, L. Arcee trinity large technical report, 2026. URL https://github.com/arcee-ai/trinity-large-tech-report.

Tang, Y., Li, X., Liu, F., Guo, W., Zhou, H., Wang, Y., Han, K., Yu, X., Li, J., Zang, H., Mi, F., Meng, X., Liu, Z., Chen, H., Zheng, B., Chen, C., Yan, Y., Tang, R., Qin, P., Chen, X., Tao, D., and Wang, Y. Pangu pro moe: Mixture of grouped experts for efficient sparsity, 2025. URL https://arxiv.org/abs/2505.21411.

Team, M. L., Bayan, Li, B., Lei, B., Wang, B., Rong, B., Wang, C., Zhang, C., Gao, C., Zhang, C., Sun, C., Han, C., Xi, C., Zhang, C., Peng, C., Qin, C., Zhang, C., Chen, C., Wang, C., Ma, D., Pan, D., Bu, D., Zhao, D., Kong, D., Liu, D., Huo, F., Li, F., Zhang, F., Dong, G., Liu, G., Xu, G., Li, G., Tan, G., Lin, G., Jing, H., Fu, H., Yan, H., Wen, H., Zhao, H., Liu, H., Shi, H., Hao, H., Tang, H., Lv, H., Su, H., Li, J., Liu, J., Li, J., Yang, J., Wang, J., Yang, J., Tan, J., Sun, J., Zhang, J., Fu, J., Yang, J., Hu, J., Qin, J., Wang, J., He, J., Kuang, J., Mei, J., Liang, K., He, K., Zhang, K., Wang, K., He, K., Gao, L., Shi, L., Ma, L., Qiu, L., Kong, L., Si, L., Lyu, L., Guo, L., Yang, L., Yan, L., Xia, M., Gao, M., Zhang, M., Zhou, M., Shen, M., Tuo, M., Zhu, M., Li, P., Pei, P., Zhao, P., Jia, P., Sun, P., Gu, Q., Li, Q., Li, Q., Huang, Q., Duan, Q., Meng, R., Weng, R., Shao, R., Li, R., Wu, S., Liang,

S., Wang, S., Dang, S., Fang, T., Li, T., Chen, T., Bai, T., Zhou, T., Xie, T., He, W., Huang, W., Liu, W., Shi, W., Wang, W., Wu, W., Zhao, W., Zan, W., Shi, W., Nan, X., Su, X., Li, X., Mei, X., Ji, X., Xi, X., Huang, X., Li, X., Fu, X., Liu, X., Wei, X., Cai, X., Chen, X., Liu, X., Li, X., Shi, X., Li, X., Wang, X., Chen, X., Hu, X., Miao, X., He, X., Zhang, X., Hao, X., Cao, X., Cai, X., Yang, X., Feng, Y., Bai, Y., Chen, Y., Yang, Y., Huo, Y., Sun, Y., Lu, Y., Zhang, Y., Zang, Y., Zhai, Y., Li, Y., Yin, Y., Lv, Y., Zhou, Y., Yang, Y., Xie, Y., Sun, Y., Zheng, Y., Wei, Y., Qian, Y., Liang, Y., Tai, Y., Zhao, Y., Yu, Z., Zhang, Z., Yang, Z., Zhang, Z., Xia, Z., Zou, Z., Zeng, Z., Su, Z., Chen, Z., Zhang, Z., Wang, Z., Jiang, Z., Zhao, Z., Wang, Z., and Su, Z. Longcat-flash technical report, 2025. URL https://arxiv.org/abs/2509.01322.

Wang, L., Gao, H., Zhao, C., Sun, X., and Dai, D. Auxiliary-loss-free load balancing strategy for mixture-of-experts, 2024. URL https://arxiv.org/abs/2408.15664.

Xue, F., Zheng, Z., Fu, Y., Ni, J., Zheng, Z., Zhou, W., and You, Y. Openmoe: An early effort on open mixture-of-experts language models, 2024. URL https://arxiv.org/abs/2402.01739.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025a. URL https://arxiv.org/abs/2505.09388.

Yang, J., Liu, X., Lv, W., Deng, K., Guo, S., Jing, L., Li, Y., Liu, S., Luo, X., Luo, Y., Pan, C., Shi, E., Tan, Y., Tao, R., Wu, J., Wu, X., Wu, Z., Zan, D., Zhang, C., Zhang, W., Zhu, H., Zhuo, T. Y., Cao, K., Cheng, X., Dong, J., Fang, S., Fei, Z., Guan, X., Guo, Q., Han, Z., James, J., Luo, T., Li, R., Li, Y., Liang, Y., Liu, C., Liu, J., Liu, Q., Liu, R., Loakman, T., Meng, X., Peng, C., Peng, T., Shi, J., Tang, M., Wang, B., Wang, H., Wang, Y., Xu, F., Xu, Z., Yuan, F., Zhang, G., Zhang, J., Zhang, X., Zhou, W., Zhu, H., Zhu, K., Dai, B., Liu, A., Li, Z., Lin, C., Liu, T., Peng, C., Shen, K., Qin, L., Song, S., Zhan, Z., Zhang, J., Zhang, J., Zhang, Z., and Zheng, B. From code foundation models to agents and applications: A comprehensive survey and practical guide to code intelligence, 2025b. URL https://arxiv.org/abs/2511.18538.